Plan Día por Día: 27 Enero - 10 Marzo 2026

Objetivo: Preparación completa JavaScript orientado a N8N
Duración: 42 días
Metodología: Constante, pragmática, arquitectónica

📅 SEMANA 1: FUNDAMENTOS (27 ene - 2 feb)

Martes 27 Enero - Día 1

Fase 0: Data Privacy Fundamentos

Tiempo: 5 horas

Mañana (3h):

- 09:00-10:30 → GDPR Básico
  - Lee: https://gdpr.eu/what-is-gdpr/ (español OK)
  - Focus: Qué es PII, principios clave
  - Toma notas: Consentimiento, Minimización, Derecho al olvido
- 10:30-12:00 → PII Identification
  - Lista 10 servicios que usas
  - Por cada uno: ¿Qué datos tuyos tienen? ¿Cuáles son PII?
  - Ejemplos: Gmail, LinkedIn, Banco, Instagram, etc.

Tarde (2h):

- 14:00-16:00 → Audit Personal
  - ¿Puedes eliminar tu cuenta en cada servicio?
  - ¿Has leído sus políticas de privacidad?
  - ¿Sabes dónde físicamente están tus datos?
  - Documenta en archivo `audit_personal.md`

Resultado esperado: Lista de 10 servicios con PII identificada ☑

Miércoles 28 Enero - Día 2

Fase 0: COMPLIANCE.md + Cierre

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → COMPLIANCE.md Template
  - Crea template reutilizable en Markdown:

```
# Compliance Checklist

## Datos Guardados
- [Lista]

## PII Identificada
- [Qué datos son PII]

## Retención
- [Cuánto tiempo]

## Eliminación
- [Cómo eliminar bajo petición]

## Consentimiento
- [Cómo se obtiene]
```

- 11:00-12:00 → ADR Compliance
  - Título: "Cómo manejaré PII en mis proyectos"
  - Contexto: Todos mis proyectos manejarán datos
  - Decisión: NUNCA guardar PII sin consentimiento explícito
  - Alternativas + Consecuencias

Tarde (2h):

- 14:00-16:00 → Review Fase 0 Completa
  - Repasa todo lo aprendido
  - Reflexión escrita: ¿Qué fue más difícil? ¿Qué aplicaré?
  - Verifica: ¿Tienes 1-2 ADRs escritos? ¿COMPLIANCE.md template?

Resultado esperado: Fase 0 COMPLETA ☑

Jueves 29 Enero - Día 3

Fase 0.5 + JS: Terminal + Variables

Tiempo: 5 horas

Mañana (3h): Terminal Básico

- 09:00-10:30 → Setup y comandos esenciales
  - Abre terminal (Terminal en Mac, PowerShell/CMD en Windows)
  - Practica comandos:
    - `pwd` (dónde estoy)
    - `ls` o `dir` (qué hay aquí)
    - `cd` (cambiar directorio)
    - `mkdir` (crear carpeta)
    - `touch` o `echo > file.txt` (crear archivo)
    - `rm` o `del` (eliminar - CON CUIDADO)

- 10:30-12:00 → Práctica navegación
  - Navega por tu sistema sin GUI
  - Crea estructura: `roadmap-architect/fase0/fase0.5/js/`
  - Crea archivo prueba: `test.txt`

Tarde (2h): JavaScript Variables

- 14:00-16:00 → Variables y tipos
  - Recurso: MDN "JavaScript First Steps" o JavaScript.info
  - `let`, `const`, `var` (cuándo usar cada uno)
  - Tipos: string, number, boolean, null, undefined
  - Operadores: `+`, `-`, `*`, `/`, `%`
  - `console.log()` para debugging
  - Practica: Crea archivo `dia3.js`, escribe 10 variables diferentes

Resultado esperado: Terminal funcional + Variables dominadas ☑

Viernes 30 Enero - Día 4

Fase 0.5 + JS: Git + Funciones

Tiempo: 5 horas

Mañana (3h): Git Básico

- 09:00-10:30 → Conceptos y setup
  - Qué es Git (control de versiones)
  - Instala Git: https://git-scm.com/
  - Configura: `git config --global user.name "Tu Nombre"`
  - Configura: `git config --global user.email "tu@email.com"`
- 10:30-12:00 → Comandos esenciales
  - `git init` (iniciar repo)
  - `git add .` (preparar cambios)
  - `git commit -m "mensaje"` (guardar)
  - `git status` (ver estado)
  - `git log` (ver historial)
  - Practica: Crea repo en carpeta `roadmap-architect`, commitea todo

Tarde (2h): JavaScript Funciones

- 14:00-16:00 → Funciones básicas
  - Declaración: `function nombre() {}`
  - Arrow functions: `const nombre = () => {}`
  - Parámetros y return
  - Scope básico (let vs const vs var)
  - Practica: Archivo `dia4.js`, escribe 5 funciones útiles
    - Ej: sumar, restar, saludar, calcular edad, etc.

Resultado esperado: Git local funcional + Funciones dominadas ☑

Sábado 31 Enero - Día 5

Fase 0.5 + JS: Instalaciones + Condicionales

Tiempo: 5 horas

Mañana (3h): Instalaciones Críticas

- 09:00-10:00 → PostgreSQL
  - Descarga: https://www.postgresql.org/download/
  - Instala PostgreSQL + pgAdmin 4
  - Verifica instalación: Abre pgAdmin
  - Crea database: `learning_db`
  - Crea tabla test: `users` (id, name, email)
- 10:00-11:00 → Docker Desktop
  - Descarga: https://www.docker.com/products/docker-desktop
  - Instala Docker Desktop
  - Verifica: `docker --version` en terminal
  - Primer container: `docker run hello-world`
- 11:00-12:00 → VSCode
  - Descarga: https://code.visualstudio.com/
  - Instala extensiones:
    - Prettier (formateo)
    - ESLint (linting)
    - Live Server (preview)
    - GitLens (Git)
  - Configura settings básicos

Tarde (2h): JavaScript Condicionales

- 14:00-16:00 → If/Else y lógica
  - `if`, `else if`, `else`
  - Operadores: `===`, `!==`, `>`, `<`, `>=`, `<=`
  - Lógicos: `&&`, `||`, `!`
  - Ternario: `condition ? true : false`
  - Practica: `dia5.js`, 5 ejercicios de lógica

Resultado esperado: PostgreSQL + Docker + VSCode funcionales ☑

Domingo 1 Febrero - Día 6

Fase 0.5 + JS: GitHub + Loops

Tiempo: 2 horas

Mañana (2h):

- 09:00-10:00 → GitHub Setup
  - Crea cuenta: https://github.com
  - Crea primer repositorio: `roadmap-architect`
  - Conecta repo local a GitHub:
    - `git remote add origin [URL]`
    - `git push -u origin main`
  - Verifica que todo subió correctamente
- 10:00-11:00 → JavaScript Loops
  - `for` loop
  - `while` loop
  - `for...of` (para arrays)
  - `break` y `continue`
  - Practica: `dia6.js`, 3 ejercicios con loops

Resultado esperado: Fase 0.5 COMPLETA + JS básico listo ☑

📅 SEMANA 2: JS FUNDAMENTOS (2-8 feb)

Lunes 2 Febrero - Día 7

JS: Arrays Introducción

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → Arrays Básico
  - Recurso: MDN "Arrays" + JavaScript.info
  - Crear arrays: `[]`
  - Acceso: `array[0]`
  - Métodos básicos: `.push()`, `.pop()`, `.shift()`, `.unshift()`
  - `.length`, `.includes()`, `.indexOf()`
- 11:00-12:00 → Iteración
  - `for` loop sobre arrays
  - `for...of`
  - `.forEach()` introducción

Tarde (2h):

- 14:00-16:00 → Práctica
  - Ejercicios Exercism.io: "Array" section
  - Crea `dia7.js`: gestión de lista de tareas
  - Operaciones: añadir, eliminar, buscar, listar

Resultado esperado: Arrays básicos dominados ☑

Martes 3 Febrero - Día 8

JS: Arrays `.map()` profundo

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → `.map()` completo
  - Qué hace: transforma cada elemento
  - Sintaxis: `array.map(item => newItem)`
  - Casos de uso: transformar datos
  - Diferencia vs `forEach`
  - Ejemplos:

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(n => n * 2);

const users = [{name: 'Ana'}, {name: 'Luis'}];
const names = users.map(u => u.name);
```

- 11:00-12:00 → Práctica intensiva
  - 10 ejercicios `.map()` diferentes

Tarde (2h):

- 14:00-16:00 → Proyecto mini
  - `dia8.js`: Array de productos
  - Usa `.map()` para:
    - Añadir descuento a todos
    - Extraer solo nombres
    - Formatear para display

Resultado esperado: `.map()` sin googlear ☑

Miércoles 4 Febrero - Día 9

JS: Arrays `.filter()` profundo

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → `.filter()` completo

  - Qué hace: filtra elementos según condición
  - Sintaxis: `array.filter(item => condition)`
  - Casos de uso: búsquedas, filtros
  - Ejemplos:

```
const numbers = [1, 2, 3, 4, 5, 6];
const evens = numbers.filter(n => n % 2 === 0);

const users = [{age: 17}, {age: 25}, {age: 30}];
const adults = users.filter(u => u.age >= 18);
```

- 11:00-12:00 → Práctica intensiva
  - 10 ejercicios `.filter()` diferentes

Tarde (2h):

- 14:00-16:00 → Combinando `.map()` + `.filter()`
  - Chain methods: `array.filter().map()`
  - `dia9.js`: Lista de usuarios
  - Filtra adultos → mapea solo nombres
  - Varios ejercicios combinados

Resultado esperado: `.filter()` + chaining dominados ☑

Jueves 5 Febrero - Día 10

JS: Arrays `.find()`, `.some()`, `.every()`

Tiempo: 5 horas

Mañana (3h):

- 09:00-10:30 → `.find()`
  - Qué hace: encuentra primer elemento que cumple condición
  - Devuelve elemento o `undefined`
  - Casos de uso: búsqueda de uno específico
- 10:30-12:00 → `.some()` y `.every()`
  - `.some()`: ¿al menos uno cumple?
  - `.every()`: ¿todos cumplen?
  - Devuelven boolean
  - Casos de uso: validaciones

Tarde (2h):

- 14:00-16:00 → Práctica integrada
  - `dia10.js`: Sistema de validación

    - Usa todos los métodos vistos
    - Ejercicios: buscar usuario, verificar permisos, etc.

Resultado esperado: Métodos de búsqueda/validación dominados ☑

Viernes 6 Febrero - Día 11

JS: Objetos Básico

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → Objetos fundamentos
    - Crear objetos: `{}`
    - Propiedades: `objeto.propiedad` vs `objeto['propiedad']`
    - Añadir/modificar propiedades
    - Eliminar: `delete objeto.propiedad`
    - Objetos anidados
- 11:00-12:00 → Métodos en objetos
    - Funciones como propiedades
    - `this` básico (conceptual)
    - Ejemplos:

```
const user = {
  name: 'Ana',
  age: 25,
  greet() {
    console.log(`Hola, soy ${this.name}`);
  }
};
```

Tarde (2h):

- 14:00-16:00 → Práctica
    - `dia11.js`: Modelar objetos del mundo real
    - Crea: usuario, producto, pedido
    - Con propiedades y métodos

Resultado esperado: Objetos básicos dominados ☑

Sábado 7 Febrero - Día 12

JS: Objetos Avanzado + Destructuring

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → Object methods
  - `Object.keys(obj)` → array de keys
  - `Object.values(obj)` → array de valores
  - `Object.entries(obj)` → array de [key, value]
  - Iterar sobre objetos
- 11:00-12:00 → Destructuring
  - Sintaxis: `const {name, age} = user;`
  - Renombrar: `const {name: nombre} = user;`
  - Default values: `const {city = 'Madrid'} = user;`
  - Nested destructuring

Tarde (2h):

- 14:00-16:00 → Práctica avanzada
  - `dia12.js`: Procesar objetos complejos
  - Extraer datos con destructuring
  - Iterar con Object methods

Resultado esperado: Objetos avanzados + destructuring ☑

Domingo 8 Febrero - Día 13

JS: Arrays Avanzado `.reduce()`

Tiempo: 2 horas

Mañana (2h):

- 09:00-11:00 → `.reduce()` profundo
  - Qué hace: acumula valor
  - Sintaxis: `array.reduce((acc, item) => acc + item, initial)`
  - Casos de uso: sumar, contar, agrupar
  - Ejemplos:

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((acc, n) => acc + n, 0);
// sum = 10

const items = [{price: 10}, {price: 20}];
const total = items.reduce((acc, item) => acc + item.price,
0);
// total = 30
```

  - Práctica: 5 ejercicios `.reduce()`

Resultado esperado: `.reduce()` conceptualmente claro ☑

📅 SEMANA 3: JSON + ES6+ (9-16 feb)

Lunes 9 Febrero - Día 14

JS: JSON Introducción

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → JSON Fundamentos
  - Qué es JSON (JavaScript Object Notation)
  - JSON vs Objeto JS (diferencias)
  - Sintaxis válida JSON:
    - Comillas dobles obligatorias
    - No trailing commas
    - No comentarios
  - `JSON.parse()` → string a objeto
  - `JSON.stringify()` → objeto a string
- 11:00-12:00 → Práctica conversión
  - Convertir objetos a JSON y viceversa
  - Manejar errores de parsing

Tarde (2h):

- 14:00-16:00 → JSON estructuras
  - Arrays en JSON
  - Objetos anidados en JSON
  - `dia14.js`: Lee JSON, modifica, guarda

Resultado esperado: JSON básico dominado ☑

Martes 10 Febrero - Día 15

JS: JSON Nested Navigation

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → Navegación profunda
  - Acceso a propiedades nested: `obj.level1.level2.level3`
  - Optional chaining preview: `obj?.level1?.level2`
  - Arrays dentro de objetos: `obj.items[0].name`
  - Objetos dentro de arrays: `array[0].property`

- 11:00-12:00 → Práctica estructuras complejas
    - JSON con 3-4 niveles de profundidad
    - Extraer datos específicos

Tarde (2h):

- 14:00-16:00 → Transformación JSON
    - `dia15.js`: JSON complejo (tipo API response)
    - Extrae datos específicos
    - Transforma estructura
    - Usa `.map()`, `.filter()` sobre datos JSON

Resultado esperado: JSON nested dominado ☑

Miércoles 11 Febrero - Día 16

JS: Arrays de Objetos (Integración)

Tiempo: 5 horas

Mañana (3h):

- 09:00-12:00 → Arrays de objetos profundo
    - Estructura más común en APIs
    - Operaciones combinadas:

```
const users = [
  {name: 'Ana', age: 25, city: 'Madrid'},
  {name: 'Luis', age: 30, city: 'Barcelona'},
  {name: 'María', age: 22, city: 'Madrid'}
];

// Filtrar + Mapear
const madridNames = users
  .filter(u => u.city === 'Madrid')
  .map(u => u.name);

// Encontrar
const luis = users.find(u => u.name === 'Luis');

// Acumular
const totalAge = users.reduce((sum, u) => sum + u.age, 0);
```

Tarde (2h):

- 14:00-16:00 → Proyecto mini
    - `dia16.js`: Dataset de productos (JSON)
    - Operaciones:
        - Filtrar por categoría

    - Mapear para display
    - Calcular total precio
    - Encontrar más caro/barato

Resultado esperado: Arrays de objetos fluidos ☑

Jueves 12 Febrero - Día 17

JS: ES6+ Spread Operator

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → Spread `...` profundo
    - Arrays: `[...array1, ...array2]`
    - Copiar arrays: `const copy = [...original];`
    - Objetos: `{...obj1, ...obj2}`
    - Copiar objetos: `const copy = {...original};`
    - Override properties: `{...user, age: 30}`
- 11:00-12:00 → Casos de uso
    - Combinar datos
    - Inmutabilidad (no modificar original)
    - Añadir elementos sin mutar

Tarde (2h):

- 14:00-16:00 → Rest parameters
    - Sintaxis: `function(...args) {}`
    - Diferencia spread vs rest
    - Práctica: `dia17.js`
    - Funciones con número variable de argumentos

Resultado esperado: Spread + Rest dominados ☑

Viernes 13 Febrero - Día 18

JS: ES6+ Template Literals + Optional Chaining

Tiempo: 5 horas

Mañana (3h):

- 09:00-10:30 → Template Literals
    - Sintaxis: `` `string ${variable}` ``
    - Multi-line strings
    - Expresiones dentro: `` `Total: ${price * quantity}` ``

    - Casos de uso N8N: `{{ $json.field }}`

- 10:30-12:00 → Optional Chaining
    - Sintaxis: `obj?.property`
    - Evita errores con `undefined`/`null`
    - Arrays: `array?.[0]`
    - Funciones: `obj?.method?.()`
    - MUY útil en N8N (datos inconsistentes)

Tarde (2h):

- 14:00-16:00 → Ternario profundo
    - Sintaxis: `condition ? valueIfTrue : valueIfFalse`
    - Nested ternarios (con moderación)
    - Casos de uso: asignación condicional
    - Práctica: `dia18.js`

Resultado esperado: ES6+ features N8N-críticos ☑

Sábado 14 Febrero - Día 19

Consolidación Bloque 1: Repaso Pre-Examen

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → Repaso arrays
    - Revisa todos los métodos: map, filter, find, reduce
    - Ejercicios integradores
    - Identifica gaps
- 11:00-12:00 → Repaso objetos + JSON
    - Destructuring
    - Object methods
    - JSON parsing/navigation

Tarde (2h):

- 14:00-16:00 → Ejercicios integradores
    - Codewars/Exercism: nivel 6-7 kyu
    - Problemas que combinen todo
    - Dataset complejo: filtra, mapea, acumula

Resultado esperado: Confianza pre-examen ☑

Domingo 15 Febrero - Día 20

Consolidación Bloque 1: Mock Exam + Identificar Gaps

Tiempo: 1.5 horas (consolidación formal)

Mañana (1.5h):

- 09:00-10:30 → Consolidación formal
  - Repaso conceptos débiles identificados ayer
  - Si hay mock exam del curso: hazlo
  - Si no: crea tu propio test (30 preguntas)
  - Timing real
  - Revisa errores
  - Lista: "Qué repasar mañana por la mañana"

Resultado esperado: Listo para examen entry-level ☑

Lunes 16 Febrero - Día 21

🎯 **EXAMEN JS - DÍA OFF**

Tiempo: 0 horas de estudio

Por la mañana (opcional, 1-2h max):

- Repaso ligero de conceptos clave si te sientes inseguro
- NO estudies contenido nuevo
- Mantén la calma

Examen

Después del examen:

- DESCANSA
- Celebra (aprobado o no, diste el paso)
- NO programes nada técnico hoy
- Mañana empiezas Fase 3 fresco

Resultado esperado: Examen completado ☑

📅 SEMANA 4: POST-EXAMEN JS N8N-ESPECÍFICO (17-23 feb)

Martes 17 Febrero - Día 22

JS N8N: String Methods Profundo + RegEx Básico

Tiempo: 5.75 horas

Mañana (3.75h):

- 09:00-11:00 → String methods esenciales

    - `.split(separator)` → string a array
    - `.join(separator)` → array a string
    - `.trim()` → eliminar espacios
    - `.replace(old, new)` → reemplazar
    - `.toUpperCase()`, `.toLowerCase()`
    - `.slice(start, end)`
    - `.substring(start, end)`
    - `.indexOf(substring)`

- 11:00-11:45 → **RegEx Básico para N8N** ⭐ NUEVO
    - Qué es RegEx (expresiones regulares)
    - Patterns comunes N8N:
        - Email: `/^\S+@\S+\.\S+$/`
        - Teléfono: `/\d{3}-\d{3}-\d{4}/`
        - Números: `/\d+/`
        - Espacios múltiples: `/\s+/g`
    - Métodos:
        - `.test(string)` → boolean (¿coincide?)
        - `.match(regex)` → array de coincidencias
        - `.replace(regex, replacement)` → reemplazar con pattern
    - Práctica: 3 validaciones (email, teléfono, URL)

- 11:45-12:00 → Casos de uso N8N
    - Limpiar datos sucios
    - Validar inputs
    - Extraer información específica

Tarde (2h):

- 14:00-16:00 → Práctica integrada
    - `dia22.js`: Procesamiento de texto
    - Limpiar CSV data con RegEx
    - Validar emails antes de enviar
    - Extraer información de strings
    - Formatear para diferentes outputs

Resultado esperado: String manipulation + RegEx básico fluido ☑

Miércoles 18 Febrero - Día 23

JS N8N: Array Methods Complementarios + Fechas Básico

Tiempo: 6 horas

Mañana (3h):

- 09:00-11:00 → Métodos adicionales

    - `.slice(start, end)` → extraer sección
    - `.splice(start, deleteCount, ...items)` → modificar
    - `.concat(...arrays)` → combinar
    - `.flat()` → aplanar arrays nested
    - `.sort()` → ordenar
    - `.reverse()` → invertir

- 11:00-12:00 → Casos de uso N8N
    - Reorganizar datos
    - Aplanar estructuras complejas
    - Ordenar resultados

Tarde (3h):

- 14:00-15:00 → Práctica arrays
    - `dia23.js`: Data manipulation avanzado
    - Arrays nested que necesitas aplanar
    - Ordenar por múltiples criterios
    - Extraer y reorganizar
- 15:00-16:00 → **Fechas Básico (ISO 8601 + UTC)** ⭐ NUEVO
    - Qué es ISO 8601: `2026-02-18T14:30:00Z`
    - UTC vs Local time:
        - UTC = Coordinated Universal Time (estándar)
        - Local = UTC + timezone offset
        - APIs siempre usan UTC
    - JavaScript Dates:
        - `new Date()` → fecha actual
        - `new Date('2026-02-18')` → parsear string
        - `.toISOString()` → convertir a formato ISO
        - `.getTime()` → timestamp (milisegundos)
    - Casos N8N críticos:
        - Comparar fechas: `date1 > date2`
        - Formatear para APIs (siempre ISO)
        - Calcular diferencias
        - Timezone awareness (no asumir local)
    - Práctica: 3 ejercicios
        - Parsear ISO string
        - Comparar dos fechas
        - Formatear fecha actual para API

Resultado esperado: Array methods completos + Manejo profesional de fechas ☑

Jueves 19 Febrero - Día 24

JS N8N: Try/Catch + Error Handling

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → Try/Catch fundamentos
  - Sintaxis:

    ```
    try {
      // código que puede fallar
    } catch (error) {
      // manejar error
      console.error(error.message);
    } finally {
      // siempre se ejecuta
    }
    ```

  - `throw new Error('mensaje')`
  - Error types: TypeError, ReferenceError, etc.
- 11:00-12:00 → Casos de uso N8N
  - Parsear JSON que puede ser inválido
  - Acceder a propiedades que pueden no existir
  - Fallbacks cuando algo falla

Tarde (2h):

- 14:00-16:00 → Debugging técnicas
  - `console.log()` estratégico
  - `console.table()` para arrays/objetos
  - `console.error()`, `console.warn()`
  - Práctica: `dia24.js`
  - Script robusto con manejo de errores

Resultado esperado: Error handling sólido ☑

Viernes 20 Febrero - Día 25

JS N8N: Expresiones N8N-Style

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → N8N Expression Syntax
  - Revisa N8N docs: expressions
  - Template literals profundo
  - Acceso dinámico: `obj[variable]`

  - Expresiones `{{ $json.field }}`
  - Práctica simulando estructura N8N:

```
const $json = {
  user: {name: 'Ana', age: 25},
  items: [{id: 1, name: 'Product'}]
};

// Simula expresión N8N
const result = `User: ${$json.user.name}`;
```

- 11:00-12:00 → Operaciones N8N comunes
  - Transformar items
  - Filtrar basado en condiciones
  - Crear nuevos objetos

Tarde (2h):

- 14:00-16:00 → Práctica N8N-style
  - `dia25.js`: Simula workflow N8N
  - Data que entra → procesa → output
  - Usa todos los métodos vistos
  - Piensa en "nodes" mentalmente

Resultado esperado: Lógica N8N internalizada ☑

Sábado 21 Febrero - Día 26

JS N8N: Data Transformation Challenges

Tiempo: 5 horas

Mañana (3h):

- 09:00-12:00 → Ejercicios complejos
  - Descarga datasets JSON (Kaggle/GitHub)
  - Ejercicio 1: Transform estructura A a estructura B
  - Ejercicio 2: Filtrar + agrupar + acumular
  - Ejercicio 3: Nested data a flat data
  - Ejercicio 4: Combinar múltiples sources

Tarde (2h):

- 14:00-16:00 → Review y optimización
  - Revisa tus soluciones
  - ¿Hay forma más eficiente?
  - ¿Código legible?
  - Refactoriza

Resultado esperado: Confianza en transformaciones complejas ☑

Domingo 22 Febrero - Día 27

JS N8N: Mini-Workflows en JS

Tiempo: 2 horas

Mañana (2h):

- 09:00-11:00 → Simula workflows N8N
  - `dia27.js`: 3 mini-workflows
  - Workflow 1: Trigger → Filter → Transform → Output
  - Workflow 2: Fetch data → Process → Conditional → Action
  - Workflow 3: Multiple sources → Merge → Transform
  - Usa funciones para simular "nodes"

Resultado esperado: Pensamiento en workflows ☑

📅 SEMANA 5: APIs + JSON INTEGRADO (23 feb - 1 mar)

Lunes 23 Febrero - Día 28

APIs: Conceptos Fundacionales

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → REST APIs conceptual
  - Qué es API (Application Programming Interface)
  - REST (Representational State Transfer)
  - Cliente-Servidor
  - Stateless
  - URL structure: `https://api.example.com/v1/users/123`
  - Endpoints
- 11:00-12:00 → Métodos HTTP
  - GET: obtener datos
  - POST: crear datos
  - PUT: actualizar completo
  - PATCH: actualizar parcial
  - DELETE: eliminar
  - Cuándo usar cada uno

Tarde (2h):

- 14:00-16:00 → Anatomía Request/Response

    - Request: URL + Method + Headers + Body
    - Response: Status + Headers + Body
    - Ejemplo real desglosado
    - Documentación: cómo leer API docs

Resultado esperado: REST conceptual claro ☑

Martes 24 Febrero - Día 29

APIs: Status Codes + Headers

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → Status Codes profundo
    - 2xx Success:
        - 200 OK
        - 201 Created
        - 204 No Content
    - 4xx Client Error:
        - 400 Bad Request
        - 401 Unauthorized
        - 403 Forbidden
        - 404 Not Found
    - 5xx Server Error:
        - 500 Internal Server Error
        - 503 Service Unavailable
    - Qué hacer con cada código
- 11:00-12:00 → Headers importantes
    - Content-Type: `application/json`
    - Authorization: `Bearer token`
    - Accept
    - Custom headers

Tarde (2h):

- 14:00-16:00 → Postman/Thunder Client
    - Instala Postman o Thunder Client (VSCode)
    - Primer request GET
    - Explorar response
    - Modificar headers
    - Practicar con JSONPlaceholder

Resultado esperado: HTTP profundo + tool funcional ☑

Miércoles 25 Febrero - Día 30

APIs: Autenticación

Tiempo: 5 horas

Mañana (3h):

- 09:00-10:30 → API Keys
  - Qué son
  - Dónde van (header, query param)
  - Ejemplo: `?api_key=abc123`
  - Seguridad básica
- 10:30-12:00 → Bearer Tokens
  - Qué son (JWT conceptual)
  - Header: `Authorization: Bearer <token>`
  - Expiración
  - Refresh tokens (conceptual)

Tarde (2h):

- 14:00-16:00 → OAuth2 conceptual
  - Qué problema resuelve
  - Flow básico (sin implementar)
  - Cuándo se usa
  - Ejemplos: Google, GitHub OAuth

Resultado esperado: Autenticación clara ☑

Jueves 26 Febrero - Día 31

APIs: Práctica con APIs Públicas

Tiempo: 5 horas

Mañana (3h):

- 09:00-12:00 → Consumir APIs reales
  - API 1: JSONPlaceholder (testing)
    - GET posts
    - GET users
    - POST new post
  - API 2: OpenWeatherMap
    - Consigue API key
    - GET weather por ciudad
  - API 3: REST Countries
    - GET countries
    - Filter por región

Tarde (2h):

- 14:00-16:00 → Documentar respuestas
  - Analiza estructura JSON de cada API
  - Identifica patterns comunes
  - `dia31.md`: Notas sobre cada API
  - ¿Qué tienen en común? ¿Qué difiere?

Resultado esperado: Experiencia con APIs reales ☑

Viernes 27 Febrero - Día 32

APIs: `fetch()` en JavaScript

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → `fetch()` fundamentos
  - Sintaxis básica:

    ```
    fetch('https://api.example.com/data')
      .then(response => response.json())
      .then(data => console.log(data))
      .catch(error => console.error(error));
    ```

  - GET request
  - Handling response
  - Error handling
- 11:00-12:00 → `fetch()` POST
  - Sintaxis:

    ```
    fetch('url', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(data)
    })
    ```

Tarde (2h):

- 14:00-16:00 → Práctica
  - `dia32.js`: Script que consume API
  - Usa JSONPlaceholder

    - GET → procesa → POST

Resultado esperado: `fetch()` funcional ☑

Sábado 28 Febrero - Día 33

APIs: Async/Await Básico

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → Async/Await sintaxis
    - Sintaxis más limpia que `.then()`:

```
async function getData() {
  try {
    const response = await fetch('url');
    const data = await response.json();
    return data;
  } catch (error) {
    console.error(error);
  }
}
```

    - `async` keyword
    - `await` keyword
    - Error handling con try/catch
- 11:00-12:00 → Cuándo usar async/await
    - Código más legible
    - Error handling más natural
    - Debugging más fácil

Tarde (2h):

- 14:00-16:00 → Refactor
    - Toma scripts anteriores con `.then()`
    - Refactoriza a `async/await`
    - Compara legibilidad

Resultado esperado: Async/await claro ☑

Domingo 1 Marzo - Día 34

APIs: Integración JSON + APIs

Tiempo: 2 horas

Mañana (2h):

- 09:00-11:00 → Proyecto mini integrador
  - `dia34.js`: Consume 2 APIs
  - Combina datos
  - Transforma con métodos vistos
  - Output JSON estructurado
  - Ejemplo: Weather + Country data → combined info

Resultado esperado: APIs + JSON fluido ☑

📅 SEMANA 6: PROYECTO + CONSOLIDACIÓN (2-9 mar)

Lunes 2 Marzo - Día 35

PROYECTO: Data Transformation Pipeline - Día 1

Tiempo: 5 horas

Proyecto Spec:

- **Input:** JSON complejo (array de objetos con nested data)
- **Process:** Múltiples transformaciones
- **Output:** JSON estructurado diferente
- **Tech:** Solo JavaScript (simula N8N logic)

Hoy (Setup + Core):

- 09:00-10:00 → Setup proyecto
  - Carpeta `data-transformation-pipeline/`
  - README.md con spec
  - Busca dataset JSON complejo (Kaggle o crea mock)
- 10:00-12:00 → Core logic
  - Función principal de transformación
  - Input validation
  - Estructura básica output

Tarde (2h):

- 14:00-16:00 → Transformaciones básicas
  - Implementa 2-3 transformaciones simples
  - Usa map/filter

Resultado: Proyecto iniciado ☑

Martes 3 Marzo - Día 36

PROYECTO: Data Transformation Pipeline - Día 2

Tiempo: 5 horas

Mañana (3h):

- 09:00-12:00 → Transformaciones avanzadas
  - Nested data manipulation
  - Reduce para acumulaciones
  - Conditional transformations
  - Edge cases

Tarde (2h):

- 14:00-16:00 → Error handling
  - Try/catch en puntos críticos
  - Validación de data
  - Fallbacks cuando data incompleta

Resultado: Proyecto funcional ☑

Miércoles 4 Marzo - Día 37

PROYECTO: Data Transformation Pipeline - Día 3

Tiempo: 5 horas (incluye buffer proyecto)

Mañana (3h):

- 09:00-11:00 → Testing y refinamiento
  - Test con diferentes inputs
  - Identificar bugs
  - Fix issues
- 11:00-12:00 → Optimización
  - ¿Código legible?
  - ¿Puedes simplificar algo?
  - Comentarios donde necesario

Tarde (2h):

- 14:00-16:00 → Documentation
  - README completo:
    - Qué hace
    - Cómo usar
    - Input format expected
    - Output format
    - Ejemplos
  - Comments en código complejo

Resultado: Proyecto completo + documentado ☑

Jueves 5 Marzo - Día 38

GitHub Profesional Setup

Tiempo: 5 horas

Mañana (3h):

- 09:00-11:00 → Organizar repos
  - Clean up repos existentes
  - Elimina tests/basura
  - Estructura clara por proyecto
- 11:00-12:00 → READMEs profesionales
  - Cada proyecto con README:
    - Título
    - Descripción
    - Tech stack
    - Cómo usar
    - Screenshots/ejemplos si aplica

Tarde (2h):

- 14:00-16:00 → Profile README
  - Crea repo `tu-username/tu-username`
  - README.md personalizado:
    - Breve intro
    - Skills principales
    - Proyectos destacados
    - Contacto
  - Minimalista pero profesional

Resultado: GitHub presentable ☑

Viernes 6 Marzo - Día 39

GitHub: Commits y Pinning

Tiempo: 3 horas

Mañana (3h):

- 09:00-10:30 → Commits descriptivos
  - Revisa commit history
  - Si es confuso: squash/rewrite (Git avanzado opcional)

    - Aprende buenas prácticas:
        - `feat: add user authentication`
        - `fix: resolve date parsing bug`
        - `docs: update README with API examples`

- 10:30-12:00 → Pinned repos
    - Selecciona 3-4 mejores proyectos
    - Pin en perfil
    - Verifica que READMEs están OK
    - Descripción corta en cada repo

Resultado: GitHub profesional mínimo ☑

Sábado 7 Marzo - Día 40

ADR Final: JavaScript Orientado a N8N

Tiempo: 3 horas

Mañana (3h):

- 09:00-12:00 → Escribe ADR completo
    - Título: "JavaScript Orientado a N8N: Decisiones de Aprendizaje"
    - Contexto:
        - Objetivo: N8N productivo
        - Timeline: 42 días
        - Restricción: entry-level a N8N-ready
    - Decisión:
        - Qué JS aprendí (y por qué)
        - Qué JS omití (y por qué)
        - Orden de aprendizaje elegido
    - Alternativas:
        - Podría haber aprendido más DOM/Events
        - Podría haber profundizado en OOP
        - Podría haber hecho 3 proyectos
    - Trade-offs:
        - Gané: JS N8N-específico sólido
        - Perdí: Profundidad en temas no-N8N
    - Consecuencias:
        - Listo para N8N
        - Tendré que aprender X cuando llegue a Fase 2
    - Validación:
        - Proyecto completado
        - Conceptos internalizados
        - Confianza en transformaciones

Resultado: ADR escrito ☑

Domingo 8 Marzo - Día 41

Consolidación Bloque 2: Repaso Completo

Tiempo: 1.5 horas (consolidación formal)

Mañana (1.5h):

- 09:00-10:30 → Repaso fase 3
  - Revisa conceptos clave:
    - Arrays, objetos, JSON
    - APIs, fetch, async/await
    - ES6+ features
    - String/array methods
  - Ejercicio integrador final:
    - Consume API
    - Transforma data
    - Output estructurado
  - Identifica últimos gaps

Resultado: Conocimientos consolidados ☑

Lunes 9 Marzo - Día 42

Preparación Final N8N

Tiempo: 3 horas

Mañana (3h):

- 09:00-10:00 → Repaso express
  - Revisa notas clave
  - Conceptos que usarás más en N8N:
    - map, filter, find
    - JSON navigation
    - Template literals
    - Optional chaining
- 10:00-11:00 → N8N Installation (si no hecho)
  - Instala N8N: `npm install n8n -g`
  - O Docker: `docker run -p 5678:5678 n8nio/n8n`
  - Verifica que abre: `localhost:5678`
- 11:00-12:00 → Review N8N docs
  - Lee: N8N documentation "Getting Started"
  - Familiarízate con interface
  - Lee: "Code Node" documentation
  - Ejemplos de expresiones

Resultado: Listo para empezar N8N mañana ☑

📅 MARTES 10 MARZO - DÍA N8N

HOY NO ES PARTE DEL PLAN

Hoy empieza tu nueva fase: **N8N Productivo**

Tienes:

- ☑ JavaScript sólido orientado a N8N
- ☑ JSON/APIs dominados
- ☑ 1 proyecto en portfolio
- ☑ GitHub profesional
- ☑ Pensamiento arquitectónico (ADRs)
- ☑ Error handling robusto
- ☑ 42 días de aprendizaje constante

Estás listo. 🚀

📊 RESUMEN FINAL

Distribución Real de Horas:

Fase	Horas Planificadas	Días
Fase 1 (Fundamentos)	25h	6 días
Fase 2 (Pre-examen)	75h	15 días
Examen	0h	1 día OFF
Fase 3 (Post-examen + Proyecto)	78h	22 días
TOTAL	**178h**	**42 días**

Buffer disponible: 27h (~5 días completos)

Checkpoints Cumplidos:

☑ **Fase 1 (31 ene):**

- Fase 0 completa
- Terminal/Git/GitHub funcional
- PostgreSQL/Docker instalados
- JS básico dominado

☑ **Fase 2 (15 feb):**

- Arrays avanzados sin googlear

- Objetos + destructuring natural
- JSON nested fluido
- Entry-level ready

☑ **Fase 3 (10 mar):**

- Proyecto completado
- GitHub profesional
- APIs fluidas
- N8N-ready

Recursos Utilizados:

JS:

- MDN Web Docs
- JavaScript.info
- Exercism.io

APIs:

- Postman/Thunder Client
- JSONPlaceholder
- OpenWeatherMap
- REST Countries

Herramientas:

- Terminal
- Git/GitHub
- VSCode
- PostgreSQL
- Docker
- N8N

🎯 PRÓXIMO PASO

Mañana 27 enero, Día 1:

1. ☕ Desayuna tranquilo
2. 📖 Lee GDPR básico (1.5h)
3. 📝 Audit PII personal (1.5h)
4. 💻 Empieza con confianza

Tienes 42 días. Vas a lograrlo. ☑

Este es tu roadmap. Guárdalo. Síguelo. Ajusta cuando necesario. Documenta cuando decidiste.

Arquitecto en formación, no ejecutor ciego.

🚀